workout app 1.0

- exercise
    - ID → auto generated UUID
    - name
        - user set name for the exercise
        - ej: bench press; chest fly; bicep curl
    - aliases
        - list of aliases for the exercise
        - ej: around the world, dumbbell halos
    - notes
        - description or notes for the exercise
        - ej: left shoulder hurts, good progress, remember to keep knees close

exercise metrics

From what I gather an exercise has a number of sets and reps and associated weight. This is crucial. I want to account for any funky stuff done here. So we get:

- SetTemplate(parent)
    - UUID
    - set(child)
        - UUID
        - number of reps
        - weight
            - unit (kg or lbs)

volume = sum of (reps × weight) across all in a SetTemplate. This value is adimensional

An exercise has sets. Which is a list of set(child) associated with SetTemplate(parent) which are associated with the exercise. Each set has its number of reps and the weight.

The log entry stores a full copy of the set data at the time of logging, independent of any future edits to the SetTemplate.

note: an alias can be used in order to find exercises under different names. In the UI

<u>tracking progress</u>:

when creating an exercise you will establish the SetTemplate(parent) attribute. Additionally you will log in that exercise on a specified date (dd/MM/YY).

SetTemplate(parent) will then have

- edit button
    - will let you edit everything on that set. Add or delete set(child) and modify the values of each set(child) so weight, the unit of weight and the number of reps
- update/log button

        - same as edit but enables a “log” button. You update your info and an update button is enabled. Once you press it you are prompted for a date (dd/MM/YY) the info on the exercise is updated and logged

        -

- workout
  a set of exercises
    - UUID
    - name
        - name describing the workout (leg day 1, push/pull day, chest, etc…)
    - list of exercises in the workout
    - notes
        - user notes with whatever they want to add

- workout plan
    - UUID
    - set of workouts

For now I’ve seen most plans are divided into weeks. So per week you have chest day, arm day, leg day, general stuff or whatever. I will organize this for weeks.

However I want to keep things modular so if I ever want to add the ability to organize per 3 day batches, months or years you could do that. Not a priority at all though

    - name
    - list of workouts
        - each workout in the list has
            - day (day of the week it will be done)
                - one day can contain multiple workouts

==========

Body measurements

An important part of workout progress is tracking body measurements. The app should provide a flexible system to track different types of measurements rather than fix sets. It should be easy for beginners and advanced lifters. In that sense the app default will only have weight as a default measurement to track. However, the user can CRUD any measurements at any time.

In that sense we will have

- Measurement
    - unique UUID
    - name
    - entry list
        - list of entries

- Entry
    - UUID
    - value
        - unit
            - ej: cm, kg, lb, in
    - date

the user should be able to CRUD entries or measurements within his app at any time.

==========

A crucial part of the app is to log and see progress.

So in the progress track of an exercise each time the exercise is updated a snapshot of the values for the SetTemplate(parent) should be saved for that exercise. In that sense each snapshot has:

- specific exercise
    - UUID
    - date of entry
    - notes (optional)
    - SetTemplate(parent)
        - set list
            - set(child)
                - reps
                - weight
                - unit of weight

Workout Session Tracker

Users should also be able to log a workout session. A workout session as an entry for a workout. For example, I did push/pull day on Tuesday. In that regard the user must be able to select a pre-existing workout and log it into the session tracker.

The data on a workout session log is:UUID, specific workout, date, notes (optional)

when entering this information the default day should be ‘today’

—----------

The user should also be able to track his progress through 2 views. The first one is a graph view for all entries under a specific measurement or exercise. And the other is a table view for all measurements or exercises.

The table view will have a row by row breakdown of whatever metric is being shown.

For measurement you'll have the columns: value, unit, date. For exercises you'll have 2 table views. One for weight and one for volume (amount of reps). You'll have the following columns: weight or volume , unit (not present in case of volume), date

Since measurements only hold one value (weight across time for example) there should only be one plotted line with the value on the 'y' axis and an adjustable time frame on the 'x' axis

Exercises have a weight and a volume. Users should see 2 graphed lines that can be toggled on or off. The 'y' axis should have the weight or volume while the 'x' axis has the adjustable time frame for the view. For this table weight is the logged average weight per exercise. Volume follows the sum of (reps × weight) across all in a SetTemplate for a specific logging entry.

==========
technology: flutter and dart
target system: android

==========
proposed project structure:
lib/
  models/   ← your Dart classes (Exercise, Workout, etc.)
  db/   ← SQL schema, database helper, migrations
  repositories/   ← one file per entity, all DB queries live here
  screens/   ← one folder per screen
  widgets/   ← reusable UI components
  main.dart
==========
packages:
Local storage:

sqflite — SQLite for Flutter. Your main database.
path_provider — finds the correct local directory to store the DB file. Required companion to sqflite.

Unique IDs:

uuid — generates UUID v4. You'll call this every time you create a new entity.

Charts:

fl_chart — the standard Flutter charting library. Well documented and handles line charts cleanly.

Date handling:

intl — for formatting dates as dd/MM/yy in the UI while storing ISO 8601 internally.

Utilities:

equatable — makes comparing Dart objects by value much less painful. Useful for your model classes.
==========